

PROGRAMMING FOR PROBLEM SOLVING (3110003)



Prepared By,

Prof. Shraddha Modi

Computer Engineering

LDCE, Ahmedabad

CHAPTER-8

STRUCTURE

Prepared By,
Prof. Shraddha Modi
Computer Engineering
LDCE, Ahmedabad

ROADMAP

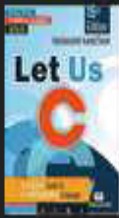
- ❑ Basics of structure,
- ❑ Structure members, accessing structure members,
- ❑ Nested structures,
- ❑ Array of structures, structure and functions,
structures and pointers

INTRODUCTION

- ▶ We need combination of various datatypes to understand different entity/object.

- ▶ **Example-1:**

- **Book**



- Title:** Let Us C

- Author:** Yashavant Kanetkar

- Page:** 320

- Price:** 255.00

- Datatype:** char / string

- Datatype:** char / string

- Datatype:** int

- Datatype:** float

- ▶ **Example-2:**

- **Student**



- Name:** ABC

- Roll_No:** 180540107001

- CPI:** 7.46

- Backlog:** 01

- Datatype:** char / string

- Datatype:** int

- Datatype:** float

- Datatype:** int

INTRODUCTION

- Structure is a collection of logically related data items of different datatypes grouped together under single name.
- Structure is a **user defined datatype**.
- Structure helps to build a complex datatype which is more meaningful than an array.
- But, an array holds similar datatype record, when structure holds different datatypes records.
- Two fundamental aspects of Structure:
 - Declaration of Structure Variable
 - Accessing of Structure Member

INTRODUCTION

❖ Definition of structure:

```
struct structure_name  
  
{  
  
    datatype member1;  
  
    datatype member2;  
  
    ...  
  
    datatype member n;  
  
};
```

❖ Declaration of structure before main:

```
struct student  
  
{  
  
    int sid;  
  
    char sname[10];  
  
};
```

INTRODUCTION – POINTS TO REMEMBER

- You must terminate structure definition with **semicolon ;**.
- You **cannot assign value** to members inside the structure definition, it will cause **compilation error**.

Example

```
struct student
{
    char name[30] = "ABC"; // Student Name
    ...
};
```

DECLARE STRUCTURE VARIABLE

- ❖ A data type defines various properties about data stored in memory.
- ❖ To use any type we must declare its variable.
- ❖ Hence, let us learn how to create our custom structure type objects also known as structure variable.
- ❖ In C programming, there are two ways to declare a structure variable:
 1. Along with structure definition
 2. After structure definition

DECLARE STRUCTURE VARIABLE

1. Declaration along with the structure definition

Syntax

```
1 struct structure_name
2 {
3     member1_declaration;
4     member2_declaration;
5     . . .
6     memberN_declaration;
7 } structure_variable;
```

Example

```
1 struct student
2 {
3     char name[30]; // Student Name
4     int roll_no; // Student Roll No
5     float CPI; // Student CPI
6     int backlog; // Student Backlog
7 } student1;
```

DECLARE STRUCTURE VARIABLE

2. Declaration after Structure definition

Syntax

```
1 struct structure_name structure_variable;
```

Example

```
1 struct student
2 {
3     char name[30]; // Student Name
4     int roll_no; // Student Roll No
5     float CPI; // Student CPI
6     int backlog; // Student Backlog
7 }
8 struct student student1; // Declare structure
   variable
```

STRUCTURE VARIABLE INITIALIZATION

```
struct student
{
char sname[10];
int sid;
};

struct student s1 = {"abc", 1};
struct student s2 = {"def", 2};
```

ACCESS STRUCTURE MEMBERS

- We must assign data to individual **structure members** separately.
- C supports two operators to access structure members, using a structure variable.
 1. Dot/period operator (.)
 2. Arrow operator (->)

ACCESS STRUCTURE MEMBERS

1. Dot/period operator (.)

- It is known as member access operator. We use **dot operator** to access members of simple structure variable.

Syntax

```
1 structure_variable.member_name;
```

Example

```
1 // Assign CPI of student1  
2 student1.CPI = 7.46;
```

ACCESS STRUCTURE MEMBERS

2. Arrow operator (->)

- In C language it is illegal to access a structure member from a pointer to structure variable using dot operator.
- We use **arrow operator** to access structure member from pointer to structure.

Syntax

```
1 pointer_to_structure->member_name;
```

Example

```
1 // Student1 is a pointer to student type  
2 student1 -> CPI = 7.46;
```

WRITE A PROGRAM TO READ AND DISPLAY STUDENT INFORMATION USING STRUCTURE.

```
#include <stdio.h>

struct student
{
    char name[40];
    int roll;
    float CPI;
    int backlog;
};

int main()
{
    struct student student1;

    printf("Enter Student Name:");
    scanf("%s", student1.name);

    printf("Enter Student Number:");
    scanf("%d", &student1.roll);

    Program Continues....
```

WRITE A PROGRAM TO READ AND DISPLAY STUDENT INFORMATION USING STRUCTURE.

```
printf("Enter Student CPI:");      printf("Student Enrollment:
scanf("%f", &student1.CPI);        %d\n", student1.roll);

printf("Enter Student Backlog:");  printf("Student CPI: %f\n",
scanf("%d", &student1.backlog);    student1.CPI);

printf("\nStudent data.");         printf("Student Backlog: %i\n",
printf("Student name: %s\n",      student1.backlog);
student1.name);                  }
```

WRITE A PROGRAM TO READ AND DISPLAY STUDENT INFORMATION USING STRUCTURE.

❖ O/P

```
Enter Student Name:shiv
Enter Student Roll Number:1
Enter Student CPI:8
Enter Student Backlog:0
```

```
Student data.
Student name: shiv
Student Enrollment: 1
Student CPI: 8.000000
Student Backlog: 0
```

ARRAY OF STRUCTURE

Syntax

```
1 struct structure_name
2 {
3     member1_declaration;
4     member2_declaration;
5     ...
6     memberN_declaration;
7 } structure_variable[size];
```

ARRAY OF STRUCTURE

```
#include<stdio.h>
```

```
struct student {
```

```
char name[20];
```

```
int rollno;
```

```
float cpi;
```

```
};
```

```
int main( ) {
```

```
int i,n;
```

```
printf("Enter how many
```

```
records u want to store : ");
```

```
scanf("%d",&n);
```

```
struct student sarr[n];
```

ARRAY OF STRUCTURE

```
for(i=0; i<n; i++)
{
    scanf("%d",&sarr[i].rollno);
    printf("Enter CPI : ");
    scanf("%f",&sarr[i].cpi);
    printf("\nEnter %d record : \n",i+1);
}
printf("Enter Name : ");
scanf("%s",sarr[i].name);
printf("Enter RollNo. : ");
printf("\n\tName\tRollNo\tMarks\t\n");
```

ARRAY OF STRUCTURE

```
for(i=0; i<n; i++) {  
    printf("\t%s\t\t%d\t\t%.2f\t\t  
n", sarr[i].name,  
sarr[i].rollno, sarr[i].cpi);  
  
}  
  
return 0;  
  
}
```

Enter how many records u want to store : 2

Enter 1 record :

Enter Name : shiv

Enter RollNo. : 1

Enter CPI : 8

Enter 2 record :

Enter Name : krish

Enter RollNo. : 2

Enter CPI : 8

Name	RollNo	Marks
shiv	1	8.00
krish	2	8.00

STRUCTURE USING POINTER

```
#include <stdio.h>
```

```
struct student
```

```
{
```

```
char name[20];
```

```
int rollno;
```

```
float cpi;
```

```
};
```

```
void main()
```

```
{
```

```
struct student *sptr, stud1;
```

```
sptr = &stud1;
```

```
Program continues...
```

STRUCTURE USING POINTER

```
printf("Enter Name: ");  
scanf("%s", sptr->name);  
printf("Enter RollNo: ");  
scanf("%d", &sptr->rollno);  
printf("Enter CPI: ");  
scanf("%f", &sptr->cpi);  
printf("\nStudent Details:\n");  
printf("Name: %s",  
       sptr->name);  
printf("RollNo: %d",  
       sptr->rollno);  
printf("\nCPI: %f", sptr->cpi);  
}
```

STRUCTURE USING POINTER

Output:

```
Enter Name: shiv
```

```
Enter RollNo: 1
```

```
Enter CPI: 7
```

```
Student Details:
```

```
Name: shiv
```

```
RollNo: 1
```

```
CPI: 7.000000
```

NESTED STRUCTURE

- When a structure contains another structure, it is called nested structure.
- For example, we have two structures named Address and Student. To make Address nested to Student, we have to define Address structure before and outside Student structure and create an object of Address structure inside Student structure.

NESTED STRUCTURE

Syntax

```
1 struct structure_name1
2 {
3     member1_declaration;
4     member2_declaration;
5     ...
6     memberN_declaration;
7 };
8 struct structure_name2
9 {
10     member1_declaration;
11     member2_declaration;
12     ...
13     struct structure1 obj;
14 };
```

NESTED STRUCTURE

READ AND DISPLAY STUDENT INFORMATION USING NESTED OF STRUCTURE

```
#include<stdio.h>
```

```
struct Address
```

```
{
```

```
char HouseNo[25];
```

```
char City[25];
```

```
char PinCode[25];
```

```
};
```

```
struct Student
```

```
{
```

```
char name[25];
```

```
int roll;
```

```
float cpi;
```

```
struct Address Add;
```

```
};
```

Program Continues...

NESTED STRUCTURE

READ AND DISPLAY STUDENT INFORMATION USING NESTED OF STRUCTURE

```
void main()
{
    int i;
    struct Student s;
    printf("\n\tEnter Student
    Name : ");
    scanf("%s",s.name);

    printf("\n\tEnter Student Roll
    Number : ");
    scanf("%d",&s.roll);
    printf("\n\tEnter Student CPI :
    ");
    scanf("%f",&s.cpi);
    Program Continues...
```

NESTED STRUCTURE

READ AND DISPLAY STUDENT INFORMATION USING NESTED OF STRUCTURE

```
printf("\n\tEnter Student House No : ");
```

```
scanf("%s",s.Add.HouseNo);
```

```
printf("\n\tEnter Student City : ");
```

```
scanf("%s",s.Add.City);
```

```
printf("\n\tEnter Student Pincode : ");
```

```
scanf("%s",s.Add.PinCode);
```

Program Continues...

NESTED STRUCTURE

READ AND DISPLAY STUDENT INFORMATION USING NESTED OF STRUCTURE

```
printf("\nDetails of Students");  
  
printf("\n\tStudent Name : %s",s.name);  
  
printf("\n\tStudent Roll Number : %d",s.roll);  
  
printf("\n\tStudent CPI : %f",s.cpi);  
  
printf("\n\tStudent House No : %s",s.Add.HouseNo);  
  
printf("\n\tStudent City : %s",s.Add.City);  
  
printf("\n\tStudent Pincode : %s",s.Add.PinCode);  
  
}
```

NESTED STRUCTURE

READ AND DISPLAY STUDENT INFORMATION USING NESTED OF STRUCTURE

❖ Output:

```
Enter Student Name : shiv
Enter Student Roll Number : 1
Enter Student CPI : 8
Enter Student House No : 403
Enter Student City : ahm
Enter Student Pincode : 384287
```

Details of Students

```
Student Name : shiv
Student Roll Number : 1
Student CPI : 8.000000
Student House No : 403
Student City : ahm
Student Pincode : 384287
```

STRUCTURE & FUNCTION

WRITE A PROGRAM TO DECLARE TIME STRUCTURE AND READ TWO DIFFERENT TIME PERIOD AND DISPLAY SUM OF IT USING FUNCTION.

```
#include<stdio.h>
```

```
struct Time {
```

```
int hours;
```

```
int minutes;
```

```
int seconds;
```

```
};
```

```
struct Time input(); // function declaration
```

STRUCTURE & FUNCTION

WRITE A PROGRAM TO DECLARE TIME STRUCTURE AND READ TWO DIFFERENT TIME PERIOD AND DISPLAY SUM OF IT USING FUNCTION.

```
int main()
{
    struct Time t;

    t=input();

    printf("Hours : Minutes : Seconds\n %d : %d : %d", t.hours,
    t.minutes, t.seconds);

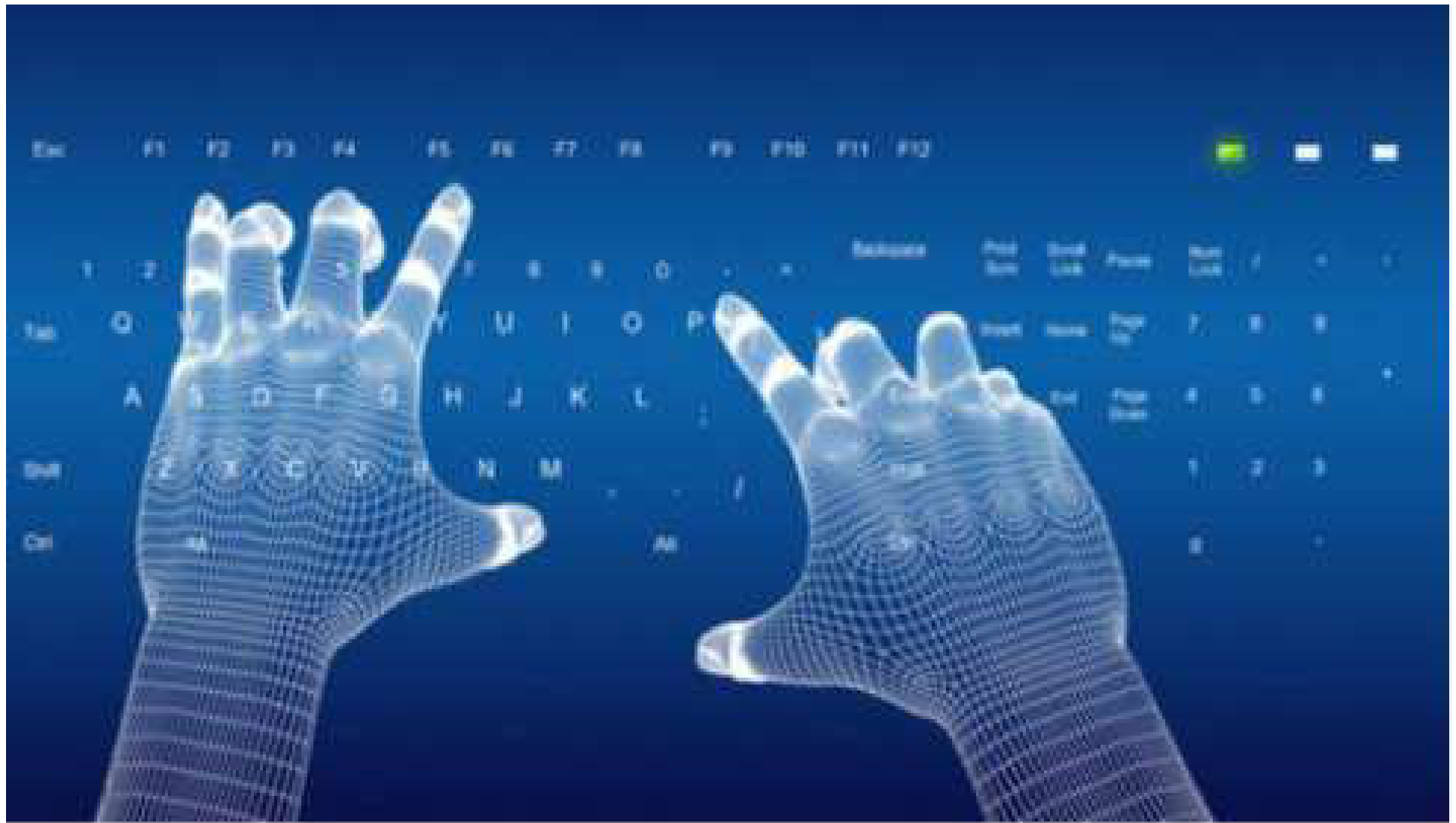
    return 0;
}
```

STRUCTURE & FUNCTION

WRITE A PROGRAM TO DECLARE TIME STRUCTURE AND READ TWO DIFFERENT TIME PERIOD AND DISPLAY SUM OF IT USING FUNCTION.

```
struct Time input()                                printf ("Enter Seconds: ");
{
                                                    scanf ("%d",&tt.seconds);
struct Time tt;                                    return tt; // return structure
                                                    variable
printf ("Enter Hours: ");
scanf ("%d",&tt.hours);                          }
printf ("Enter Minutes: ");
scanf ("%d",&tt.minutes);
```

Structure	Union
The keyword struct is used to declare structure.	The keyword union is used to declare an union.
Structure allocates the memory equal to the total memory required by the members.	allocates the memory equal to the maximum memory required by the member of the union.
In structure, each member has their own memory space.	In union, one block is used by all the member of the union.
All data members in a structure are active at time.	Only one data member is active at time.
It is commonly used in most of the applications.	It is not commonly used.
Example: <pre>struct structure1 { char c; long l; char *p; };</pre> It occupies 6 bytes of memory.	Example: <pre>union union1 { char c; long l; char *p; };</pre> It occupies 4 byte of memory, that is maximum size of union variable long l .



Thank You !